

CSE5095: Types and Programming Languages

Homework 1: Inference rules, derivations, rule induction

Prof. Stefan Muller

Out: Thursday, Sep. 3
Due: Thursday, Sep. 10 11:59pm EDT

This assignment contains 4 tasks, for a total of 25 points.

1 Logistics

Please read and follow these instructions carefully.

Writing up Solutions. Please submit your homework writeup as a .pdf or .doc document (.pdf is preferred). Typesetting your answers using LaTeX is strongly recommended, as it produces the easiest-to-read documents. Typed answers using Word or other tools, or neatly handwritten and scanned answers, are acceptable. (Note that Word, Google Docs, etc., make it unreasonably difficult to write inference rules, which you'll be doing a lot).

Collaboration and Academic Honesty. This homework is to be completed individually. See the syllabus for more detail on the collaboration policy, and in particular the policy on AI.

2 XML

In this section, we'll be working with the X is XML judgment and associated inference rules from lecture (see the notes posted on the Course Schedule on HuskyCT for reference).

Task 1 (4 points).

Add another inference rule for showing that the concatenation of 3 XML strings (e.g., XYZ) is also an XML string.

Task 2 (6 points).

As it turns out, the rule you developed above is unnecessary. Show this is the case by writing a derivation for the conclusion XYZ is XML assuming the premises of the rule you wrote in Task 1. Your derivation should use *only the rules from class*, and not the new rule you added. Make sure to include the names/numbers of the rules you're using. (As an aside, when a rule is derivable using other, already existing, rules like this, we say the rule is *admissible*.)

3 Red-Black Trees

In data structures, a red-black tree is a binary tree whose nodes are colored either red or black. Red-black trees must obey the following rules:

- Leaf nodes must be black.

- No red node has red children.
- Any path from the root to a leaf contains the same number of black nodes.

We will define a judgment $T \text{ isTree } c \ n$, meaning that T is a valid red-black tree whose root node has the color c and all of whose paths from root to leaf have exactly n black nodes. In this judgment, T , c , and n are metavariables. We will allow instantiating these metavariables as follows:

- T is a tree, defined as either **Leaf**, meaning a leaf, **BNode**(T_1, T_2), meaning a black node with children T_1 and T_2 (both of which are also, recursively, trees), or **RNode**(T_1, T_2), meaning a red node with children T_1 and T_2 .
- c is a color: either **Black** or **Red**.
- n is a natural number.

Using Backus-Naur Form, a notation we haven't seen yet as of when I'm posting this homework, but which we will see very soon (and may have already by the time you're reading this), we can represent our small language as follows:

$$\begin{aligned} \text{Trees } T &::= \text{Leaf} \mid \text{BNode}(T, T) \mid \text{RNode}(T, T) \\ \text{Colors } c &::= \text{Black} \mid \text{Red} \end{aligned}$$

The rules below instantiate define when the judgment $T \text{ isTree } c \ n$ can be derived.

$$\begin{aligned} &\frac{}{\text{Leaf isTree Black } 1} \text{ (RBT-1)} && \frac{T_1 \text{ isTree } c_1 \ n \quad T_2 \text{ isTree } c_2 \ n}{\text{BNode}(T_1, T_2) \text{ isTree Black } n+1} \text{ (RBT-2)} \\ &\frac{T_1 \text{ isTree Black } n \quad T_2 \text{ isTree Black } n}{\text{RNode}(T_1, T_2) \text{ isTree Red } n} \text{ (RBT-3)} \end{aligned}$$

Rule (RBT-1) says that leaves are valid trees as long as they are black: a leaf by definition has 1 black node on any path from root to leaf. Rule (RBT-2) allows a black node with two children as long as both children are valid red-black trees with the same number of black nodes on any path (regardless of what color their roots are); the result is a tree with a black root and $n+1$ black nodes on any path. Rule (RBT-3) enforces that red nodes do not have red children by requiring in the premises that both children have black roots.

Define $\text{Nodes}(T)$ to be the number of nodes, including leaves, in a tree:

$$\begin{aligned} \text{Nodes}(\text{Leaf}) &= 1 \\ \text{Nodes}(\text{BNode}(T_1, T_2)) &= 1 + \text{Nodes}(T_1) + \text{Nodes}(T_2) \\ \text{Nodes}(\text{RNode}(T_1, T_2)) &= 1 + \text{Nodes}(T_1) + \text{Nodes}(T_2) \end{aligned}$$

Task 3 (15 points).

Prove by rule induction that for any tree T , color $c \in \{\text{Red}, \text{Black}\}$ and $n \geq 1$, if $T \text{ isTree } c \ n$, then $\text{Nodes}(T) \geq 2^{n-1}$.

You must use rule induction, not any other proof technique.

Note: This, together with one or two other properties of red-black trees, proves that a valid red-black tree is approximately balanced.

4 One more wrap-up question

Task 4 (0 points).

How long (approximately) did you spend on this homework, in total minutes/hours of actual working time? Your honest feedback will help me with future homeworks.